Software Engineering 2
Lösungen der Übung: Retro-Seite

Dr. F.-K. Koschnick, Sybit GmbH

Copyright © 2024 Dr. F. K. Koschnick, Sybit GmbH. All rights reserved.

Aufgabe (Story): Retro-Seite, Neue Retro

Als User von RetroWeb möchte ich beim Klick auf eine Retro auf die Retroseite kommen, um dort alle Attribute der Retro in übersichtlicher Form zu sehen und diese ggf. editieren zu können. Ebenfalls möchte ich als User eine neue Retro anlegen können. Dazu soll es einen Button "Neue Retro" auf der Projektseite oberhalb der Retrotabelle geben. Bei Betätigung des Buttons öffnet sich ebenfalls die Retroseite, jedoch sind alle Felder leer.

Verifiziere, dass

- sich die Retroseite öffnet, wenn der User auf eine Retro auf der Projektseite klickt
- als Überschrift auf der Seite der Retroname steht
- alle Properties der Retro übersichtlich auf der Seite mit Label dargestellt werden
- die Felder mit den Daten der Properties editierbar sind
- es einen Link zurück auf die Home-Page und auf die Projektseite gibt
- Die Seite den Header und Footer hat, der auf der Login-Seite schon besteht. Header und Footer im body eingebunden sind, der Footer unten auf der Seite ist und beim Verkleinern der Höhe sich nicht über den Seiteninhalt schiebt (Verhalten wie bei der Login-Seite)
- nur ein eingeloggter User die Seite erreichen kann
- es einen Button „save“ gibt, der nur aktiv ist, wenn der User mindestens ein Eingabefeld geändert hat und alle Felder gefüllt sind
- bei Betätigung des Save-Buttons die geänderten Retro-Daten abgespeichert werden, falls es sich um eine neue Retro handelt, wird diese auch als neuer Datensatz abgespeichert

RetroController


```java
@RequestMapping(value = "/retro", method = RequestMethod.GET)
public String showRetro(@RequestParam long id, Map<String, Object> model, HttpServletRequest request) {
    HttpSession session = request.getSession();

    if (sessionService.isLoggedIn(session)) {
        try {
            Retro retro = retroService.getById(id);
            model.put("retro", retro);
        } catch(ResourceNotFoundException e) {
            //TODO Error-Message
            return ControllerConstants.REDIRECT_HOME;
        }
        model.put("userName", sessionService.getUserName(session));
        return ControllerConstants.RETRO;
    } else {
        return ControllerConstants.REDIRECT_LOGIN;
    }
}

@RequestMapping(value = "/retro", method = RequestMethod.POST)
public String updateRetro(@RequestParam long id, @RequestParam String name, @RequestParam String scheduled, @RequestParam(defaultValue = "false") boolean active, Map<String, Object> model, HttpServletRequest request) {
    HttpSession session = request.getSession();

    if (sessionService.isLoggedIn(session)) {

        try {
            Retro retro = retroService.getById(id);
            SimpleDateFormat format = new SimpleDateFormat("yyyy-MM-dd hh:mm:ss.S");

            retro.setName(name);
            retro.setScheduled(format.parse(scheduled));
            retro.setActive(active);
            retro = retroService.update(retro);
        } catch(ResourceNotFoundException e) {
            //TODO Fehlermeldung
            return ControllerConstants.REDIRECT_HOME;
        } catch (ParseException e) {
            //TODO Fehlermeldung
            return ControllerConstants.REDIRECT_HOME;
        }
        return ControllerConstants.REDIRECT_RETRO + "?id=" + id;
    } else {
        return ControllerConstants.REDIRECT_LOGIN;
    }
}
```

Im Retro-Entity

```java
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id = 0L;
```

```java
@RequestMapping(value = "/newretro", method = RequestMethod.GET)
public String showNewRetro(@RequestParam long id, Map<String, Object> model, HttpServletRequest request) {
    HttpSession session = request.getSession();
    if (sessionService.isLoggedIn(session)) {
        try {
            Retro retro = new Retro();
            Project project = projectService.getById(id);
            retro.setProject(project);
            model.put("retro", retro);
        } catch(ResourceNotFoundException e) {
            session.setAttribute(ControllerConstants.ERROR_MSG, e);
            return ControllerConstants.REDIRECT_HOME;
        }
        model.put("userName", sessionService.getUserName(session));
        return ControllerConstants.RETRO;
    } else {
        return ControllerConstants.REDIRECT_LOGIN;
    }
}
```

retro.jsp, project.jsp

```jsp
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<!DOCTYPE html>
<html>
    <c:import url="include/head.jsp"/>
    <body>
            <c:import url="include/header.jsp"/>
            <div class="page">
                <nav class="navbar navbar-expand-lg navbar-light bg-light">
                    <ul class="navbar-nav mr-auto">
                        <li class="nav-item active">
                            <a class="nav-link" href="home">Home</a>
                        </li>
                        <li class="nav-item active">
                            <a class="nav-link" href="project?id=${retro.project.id}">Project</a>
                        </li>
                        <li class="nav-item active">
                            <a class="nav-link" href="logout">Logout</a>
                        </li>
                    </ul>
                </nav>
                <br/>
                <h5 class="text-center">Retrospective: ${retro.name}</h5>

                <form class="form-default" name="formUpdateRetro" action="<c:url  value="/retro"/>" method="post" onsubmit="return checkUpdateRetroInputs()">
                    <table>
                        <tr>
                            <td class="align-middle">Name:</td><td class="align-middle"><input type="text" name="name" value="${retro.name}" oninput="enableButton()"/></td>
                        </tr><tr>
                            <td>Scheduled:</td><td><input type="datetime" name="scheduled" value="${retro.scheduled}" oninput="enableButton()"/></td>
                        </tr><tr>
                            <td>Is active:</td><td>
                                <c:choose>
                                    <c:when test="${retro.active}">
                                        <input type="checkbox" name="active" oninput="enableButton()" checked/>
                                    </c:when>
                                    <c:otherwise>
                                        <input type="checkbox" name="active" oninput="enableButton()"/>
                                    </c:otherwise>
                                </c:choose>
                            </td>
                        </tr><tr>
                            <td><input id="submitButton" type="submit" value="save" disabled="disabled"/></td>
                        </tr>
                    </table>
                    <input type="hidden" name="id" value="${retro.id}"/>
                    <input type="hidden" name="projectid" value="${retro.project.id}"/>
                </form>
            </div>
        <c:import url="include/footer.jsp"/>
    </body>
</html>
```

Link in project.jsp für neue Retro

```jsp
<nav class="navbar navbar-expand-lg navbar-light bg-light">
    <ul class="navbar-nav mr-auto">
        <li class="nav-item active">
            <a class="nav-link" href="home">Home</a>
        </li>
        <li class="nav-item active">
            <a class="nav-link" href="newretro?id=${project.id}">New Retro</a>
        </li>
    </ul>
</nav>
```

scriptRetro.js


```
function checkUpdateRetroInputs(){
var f = document.formUpdateRetro;
if(f.name.value == ''){
alert("Please enter a name!");
return false;
}
if(f.scheduled.value == ''){
alert("Please enter a date into the field scheduled!");
return false;
}

return true;
}

function enableButton() {
var btn = document.getElementById('submitButton');
btn.removeAttribute("disabled");
}
```

scriptRetro.js eingebunden in head.jsp

```
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <link rel="stylesheet" href="resources/css/retroweb.css" />
    <link rel="stylesheet" href="resources/css/bootstrap-4.0.0.min.css">
    <script type="text/javascript" src="resources/js/jquery-3.3.1.min.js"></script>
    <script type="text/javascript" src="resources/js/scriptProject.js"></script>
    <script type="text/javascript" src="resources/js/scriptRetro.js"></script>
    <script>
  var url = window.location.pathname;
  var filename = url.substring(url.lastIndexOf('/')+1);
  window.document.title = filename + " | RetroWeb";
    </script>
  </head>
```

Neue Methode update in RetroServiceImpl


```
@Override
  public Retro update(Retro domainObject) throws ResourceNotFoundException {
  LOG.debug("--> update");

  Retro retro = domainObject;
  retro.setUpdated(new Date());
  int result = retroRepository.update(retro.getName(), retro.isActive(),
    retro.getScheduled(), retro.getUpdated(), retro.getId());

  if(result < 1) {
  LOG.debug("<-- update, ResourceNotFoundException");
  throw new ResourceNotFoundException(RETRO, "id", domainObject.getId());
  }
  LOG.debug("<-- update");
  return retro;

  }
```

Natürlich muss diese Methode als Methodensignatur auch im Interface RetroService definiert sein.

Neue Methode save in RetroServiceImpl


```
@Override
public Retro save(Retro domainObject) throws ResourceAlreadyExistsException {
        Retro newRetro;
        LOG.debug("--> save");
        List<Retro> retros = retroRepository.findByName(domainObject.getName());
        if(retros.isEmpty()) {
                domainObject.setCreated(new Date());
                newRetro = retroRepository.save(domainObject);
        } else {
                LOG.debug("<-- save, ResourceAlreadyExistsException");
                throw new ResourceAlreadyExistsException(RETRO, "name", domainObject.getName());
        }
        LOG.debug("<-- save");
        return newRetro;
}
```

Natürlich muss diese Methode als Methodensignatur auch im Interface RetroService definiert sein.

Neue Methode update in RetroRepository


```
package de.htwg.retroweb.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.transaction.annotation.Transactional;

import de.htwg.retroweb.entities.Project;
import de.htwg.retroweb.entities.Retro;

import java.util.Date;
import java.util.List;

public interface RetroRepository extends JpaRepository<Retro, Long> {
List<Retro> findByProject(Project project);
List<Retro> findByName(String name);

@Transactional
@Modifying
@Query("update Retro r set r.name = ?1, r.active = ?2, r.scheduled = ?3, r.updated = ?4 where r.id = ?5")
int update(String name, boolean isActive, Date scheduled, Date updated, long id);
}
```

Methode update im Repository